

Documentazione Tecnica Forza Quattro

Giordano Iaquina, Paolo Donati, Lorenzo Canocchi

Febbraio 2026

Indice

1	Introduzione	2
2	Regole di Gioco	2
3	Protocollo di Comunicazione	3
3.1	Tipi di Messaggio	3
3.2	Risultato della Connessione al Server	3
3.3	Richiesta Cambio Username	3
3.4	Risposta Cambio Username	4
3.5	Richiesta Lista dei Giocatori	4
3.6	Risposta Lista dei Giocatori	4
3.7	Richiesta di Sfida	4
3.8	Risposta alla Sfida	5
3.9	Risultato della Sfida	5
3.10	Mossa	6
3.11	Esito della Mossa	6
3.12	Fine Partita	6
4	Diagramma di sequenza	7
5	Architettura del Server	8
5.1	Componenti Principali	8
5.2	Server	8
5.3	ClientHandler	8
5.4	GameSession	9
5.5	Strutture dati utilizzate nel Server	9
6	Architettura del Client	9
6.1	Componenti	10
6.2	NetworkService	10
6.3	ClientController	10
6.4	JFX Thread	11
6.5	Strutture dati utilizzate nel Client	11
6.6	Interfaccia Grafica	11

7	Controllo delle Regole di Gioco	11
7.1	Controllo del Pareggio	12
7.2	Controllo della Vittoria	12
8	Diagrammi delle Classi	12
8.1	Diagrammi delle Classi del Server	12
8.1.1	Package server	12
8.1.2	Package data	13
8.2	Diagrammi delle Classi del Client	13
8.2.1	Package client	13
8.2.2	Package data	14
8.2.3	Package application	14
8.2.4	Package application.controllers	15
9	Repository GitHub del progetto	15
10	Conclusioni	15

1 Introduzione

Il presente progetto consiste nella realizzazione di un sistema client-server multithread per il gioco Forza quattro in Java. Il sistema permette:

- Scelta dell'avversario da parte dell'utente;
- Due giocatori per partita;
- Più partite simultanee gestite separatamente;
- Gestione centralizzata tramite il server.

L'esperienza di gioco è stata resa più piacevole tramite la realizzazione di una GUI semplice ed intuitiva suddivisa in 2 sezioni:

- Lobby: permette di inviare, accettare e rifiutare le richieste di gioco;
- Partita: mostra la griglia di gioco 6x7 e permette di eseguire le mosse.

2 Regole di Gioco

L'applicazione implementa le regole classiche del Forza Quattro all'interno di una griglia di gioco digitale.

Il sistema gestisce l'alternanza dei turni tra i due giocatori, assicurando che ogni mossa venga effettuata nel momento corretto.

Il server controlla inoltre che le mosse inserite siano valide e verifica continuamente lo stato della partita per individuare eventuali condizioni di vittoria o di pareggio.

Al termine della partita, oppure in caso di disconnessione di uno dei giocatori, esso comunica l'esito finale ai client coinvolti.

3 Protocollo di Comunicazione

Il protocollo proprietario di comunicazione tra client e server da noi realizzato utilizza messaggi in formato **JSON**, un formato per lo scambio dati molto utilizzato.

Per la conversione degli oggetti in JSON e viceversa abbiamo utilizzato la libreria **Jackson**.

Il protocollo è basato su uno schema di comunicazione **Request / Response**.

3.1 Tipi di Messaggio

I principali tipi di messaggio definiti nel protocollo sono riportati nella tabella seguente.

Tipo di Messaggio
Server_Connection_Result
Change_Username_Request
Change_Username_Response
Play_List_Request
Play_List_Response
Challenge_Request
Challenge_Response
Challenge_Result
Move
Move_Response
Game_End

3.2 Risultato della Connessione al Server

Messaggio inviato dal server quando un client si connette.

```
{  
  "type" : "server_connection_result",  
  "username" : "user0011234"  
}
```

3.3 Richiesta Cambio Username

Il client richiede di cambiare il proprio username.

```
{  
  "type" : "change_username_request",  
  "new_username" : "player_01"  
}
```

3.4 Risposta Cambio Username

Il server può rispondere alla modifica dell'username:

```
{
  "type" : "change_username_response",
  "status" : "ok"
}
```

Oppure

```
{
  "type" : "change_username_response",
  "status" : "already_taken"
}
```

3.5 Richiesta Lista dei Giocatori

Il client richiede la lista dei giocatori disponibili.

```
{
  "type" : "play_list_request"
}
```

3.6 Risposta Lista dei Giocatori

Il server risponde alla richiesta della lista dei giocatori:

```
{
  "type" : "play_list_response",
  "list" : [
    {"username" : "player1", "status" : "free"},
    {"username" : "player2", "status" : "in_game"}
  ]
}
```

3.7 Richiesta di Sfida

Un giocatore può sfidare un altro giocatore tramite il seguente messaggio:

```
{
  "type" : "challenge_request",
  "username" : "player2"
}
```

3.8 Risposta alla Sfida

Un giocatore può rispondere alla sfida di un altro giocatore tramite i seguenti messaggi:

```
{
  "type" : "challenge_response",
  "username" : "player1",
  "status" : "ok"
}
```

Oppure:

```
{
  "type" : "challenge_response",
  "username" : "player1",
  "status" : "refused"
}
```

3.9 Risultato della Sfida

Il server comunica il risultato della sfida:

```
{
  "type" : "challenge_result",
  "status" : "ok",
  "first_move": "you"
}
```

Oppure

```
{
  "type" : "challenge_result",
  "status" : "refused",
  "first_move": "none"
}
```

Oppure

```
{
  "type" : "challenge_result",
  "status" : "client_not_found",
  "first_move": "none"
}
```

3.10 Mossa

Il client invia una mossa tramite:

```
{  
  "type" : "move",  
  "column" : "4"  
}
```

3.11 Esito della Mossa

Il server può rispondere alla mossa con:

```
{  
  "type" : "move_response",  
  "status" : "ok"  
}
```

Oppure

```
{  
  "type" : "move_response",  
  "status" : "invalid_move"  
}
```

Oppure

```
{  
  "type" : "move_response",  
  "status" : "not_your_turn"  
}
```

3.12 Fine Partita

Il server comunica la fine della partita tramite:

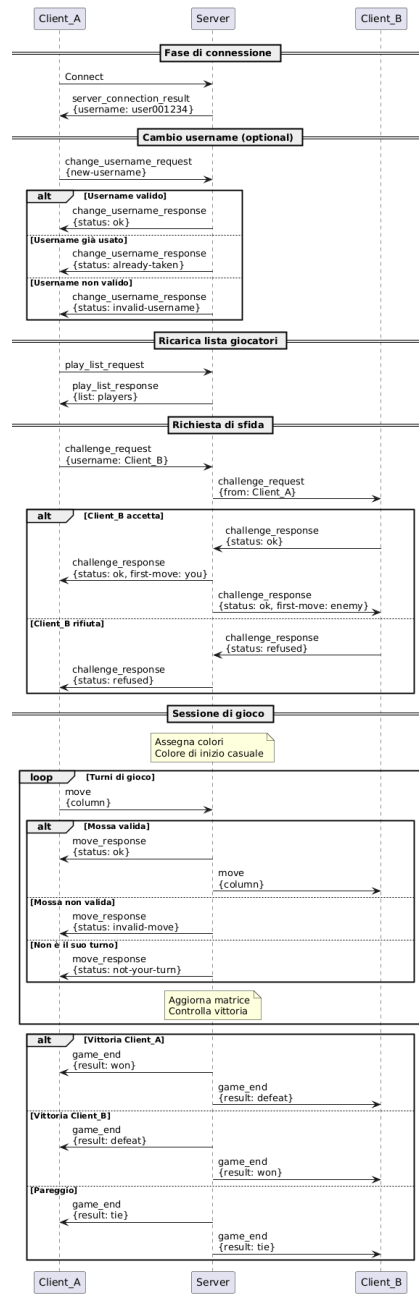
```
{  
  "type" : "game_end",  
  "result" : "won",  
  "info" : "enemy_disconnected"  
}
```

Oppure

```
{  
  "type" : "game_end",  
  "result" : "tie",  
  "info" : "game_end"  
}
```

4 Diagramma di sequenza

Il seguente diagramma mostra un esempio di come avviene lo scambio di messaggi.



5 Architettura del Server

Il server rappresenta il componente centrale del sistema e ha il compito di gestire le connessioni dei client e coordinare lo svolgimento delle partite.

L'architettura è basata su un modello multithread: per ogni client connesso viene creato un thread dedicato che permette la gestione simultanea di più utenti.

5.1 Componenti Principali

Componente	Responsabilità
Server	Avvia il server, inizializza la <code>ServerSocket</code> e gestisce l'accettazione delle connessioni dei client.
ClientHandler	Gestisce la comunicazione con un singolo client, riceve i messaggi, li elabora e invia le risposte.
ConfigLoader	Carica le impostazioni del server (porta, configurazioni di avvio).
GameSession	Rappresenta una partita di Forza Quattro e mantiene lo stato della griglia di gioco e dei giocatori.

5.2 Server

Il *Server* è il thread attivo responsabile della gestione delle nuove connessioni dei client.

Le sue principali funzioni sono:

- rimanere in ascolto sulla *ServerSocket* per eventuali nuovi client
- accettare le connessioni tramite il metodo *accept()*
- creare un nuovo *ClientHandler* per ogni client connesso
- avviare il thread dedicato al client

È presente un solo *thread* di nome *server* in ascolto sulla socket che funge da punto di ingresso per tutte le comunicazioni in arrivo.

5.3 ClientHandler

Il *ClientHandler* è un thread attivo dedicato alla comunicazione con un singolo client.

Si occupa di:

- leggere i messaggi inviati dal client tramite la socket
- elaborare le richieste ricevute dai diversi client

- gestire eventuali disconnessioni del client

Ogni client connesso ha il proprio *ClientHandler*, che garantisce comunicazioni indipendenti e isolate dagli altri client.

5.4 GameSession

Il *GameSession* rappresenta una singola partita di Forza quattro, non è un thread.

Si occupa di:

- memorizzare la griglia di gioco (6x7)
- tenere traccia dei due giocatori e del turno corrente
- validare le mosse effettuate dai giocatori
- verificare le condizioni di vittoria o pareggio

I metodi di *GameSession* vengono eseguiti dai *ClientHandler*, garantendo la consistenza dello stato della partita.

5.5 Strutture dati utilizzate nel Server

Il server utilizza strutture dati per tracciare lo stato dei client, delle partite e delle richieste di sfida.

Le principali sono:

- **Lista dei client connessi:** consente al server di memorizzare i client con il loro stato e la loro rispettiva socket.
- **Lista delle GameSession attive:** memorizza le partite in corso, con giocatori, griglia di gioco e turno corrente
- **Coda delle richieste di sfida:** contiene le sfide pendenti tra client, con mittente, destinatario e stato (in attesa / accettata / rifiutata)

Queste strutture permettono al server di gestire contemporaneamente più client e partite, garantendo coerenza e isolamento delle comunicazioni.

6 Architettura del Client

Il client rappresenta il componente dell'applicazione che consente all'utente di interagire con il sistema e di comunicare con il server. Attraverso l'interfaccia grafica l'utente può visualizzare la lobby, sfidare altri giocatori e partecipare alle partite.

L'architettura del client è stata progettata separando la gestione della rete, la gestione del client e l'interfaccia grafica, in modo da mantenere il codice modulare e facilmente estendibile.

6.1 Componenti

Componente	Responsabilità
NetworkService	Gestisce la connessione con il server, l'invio dei messaggi e la ricezione continua dei dati provenienti dalla rete tramite socket.
ClientController	Interpreta i messaggi ricevuti dal server, aggiorna lo stato del client e decide quali azioni eseguire in base al tipo di messaggio ricevuto.
ConfigLoader	Definisce porta e indirizzo IP del server a cui connettersi.
JFX Thread	Gestisce l'interazione con l'utente e aggiorna l'interfaccia grafica (JavaFX) in base alle informazioni ricevute dal controller.

6.2 NetworkService

Il *NetworkService* è il componente responsabile della comunicazione di rete tra client e server.

Le sue principali funzioni sono:

- stabilire la connessione con il server tramite socket TCP
- inviare i messaggi generati dal client
- ascoltare continuamente i messaggi provenienti dal server
- notificare il *ClientController* quando arriva un nuovo messaggio

Questo componente non contiene logica di gioco e non interagisce direttamente con l'interfaccia utente.

6.3 ClientController

Il *ClientController* rappresenta il componente centrale della logica del client. Riceve i messaggi dal *NetworkService* e decide come gestirli. In particolare si occupa di:

- interpretare i messaggi ricevuti dal server
- aggiornare lo stato del client (lobby, partita, fine partita)
- coordinare l'aggiornamento dell'interfaccia grafica
- inviare eventuali comandi al server tramite il *NetworkService*

6.4 JFX Thread

Il *JFX Thread* è responsabile della gestione dell'interfaccia grafica realizzata con JavaFX.

Attraverso questa componente l'utente può:

- visualizzare la lista dei giocatori connessi nella lobby
- inviare o ricevere richieste di sfida
- visualizzare la griglia di gioco
- effettuare le mosse durante la partita

6.5 Strutture dati utilizzate nel Client

Il client utilizza alcune strutture dati principali per gestire la comunicazione, lo stato e l'interfaccia grafica.

Le principali sono:

- **Code thread-safe** : Permette lo scambio sicuro di messaggi tra: *NetworkService*, *ClientController* e il thread dell'interfaccia grafica
- **Controller JavaFX** (*LobbyController*, *GameController*, ecc.): contengono riferimenti agli elementi grafici e gestiscono lo stato visivo dell'interfaccia

Queste strutture consentono di separare la logica di rete da quella grafica, garantendo aggiornamenti coerenti e sicuri dell'interfaccia.

6.6 Interfaccia Grafica

L'interfaccia del client è suddivisa in diverse schermate principali:

- **Lobby**: mostra la lista dei giocatori connessi e permette di inviare o ricevere richieste di sfida.
- **Game**: rappresenta la schermata di gioco con la griglia 6x7 del Forza Quattro e gli indicatori di turno.
- **EndGame**: visualizza il risultato finale della partita.
- **ServerCrash**: schermata mostrata in caso di perdita della connessione con il server.

7 Controllo delle Regole di Gioco

Il server è responsabile della validazione delle mosse e del controllo delle condizioni di vittoria.

7.1 Controllo del Pareggio

Il server mantiene un contatore delle pedine inserite nella griglia.
Dopo ogni mossa:

1. verifica se esiste una condizione di vittoria
2. se non esiste e la griglia è piena dichiara il pareggio

7.2 Controllo della Vittoria

Il controllo della vittoria viene eseguito dopo ogni mossa.

Il server analizza l'ultima pedina inserita e verifica le seguenti direzioni:

- orizzontale
- verticale
- diagonale principale
- diagonale secondaria

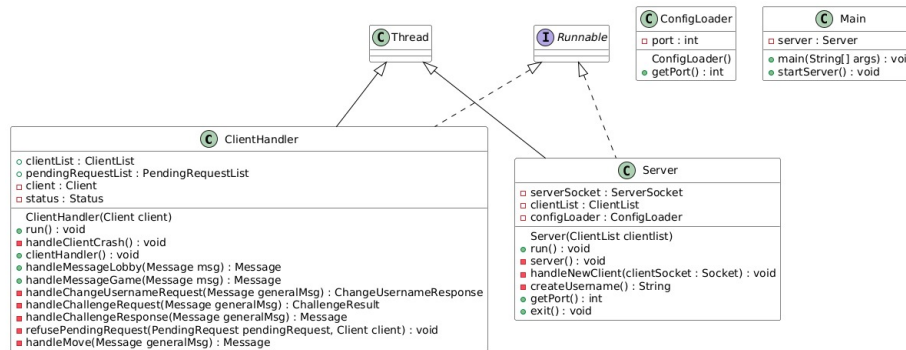
Per ogni direzione il server conta il numero di pedine consecutive dello stesso giocatore.

Se il numero di pedine consecutive raggiunge quattro, il server dichiara la vittoria del giocatore.

8 Diagrammi delle Classi

8.1 Diagrammi delle Classi del Server

8.1.1 Package server

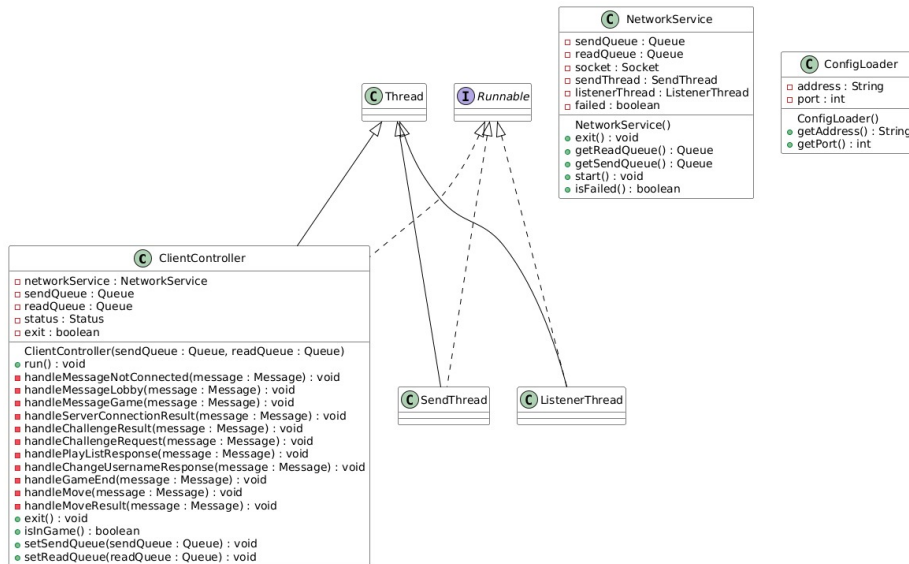


8.1.2 Package data

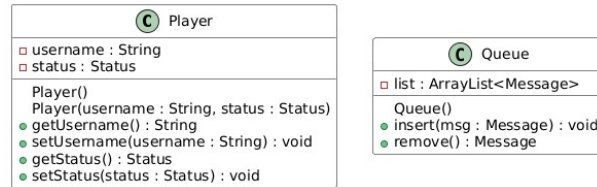


8.2 Diagrammi delle Classi del Client

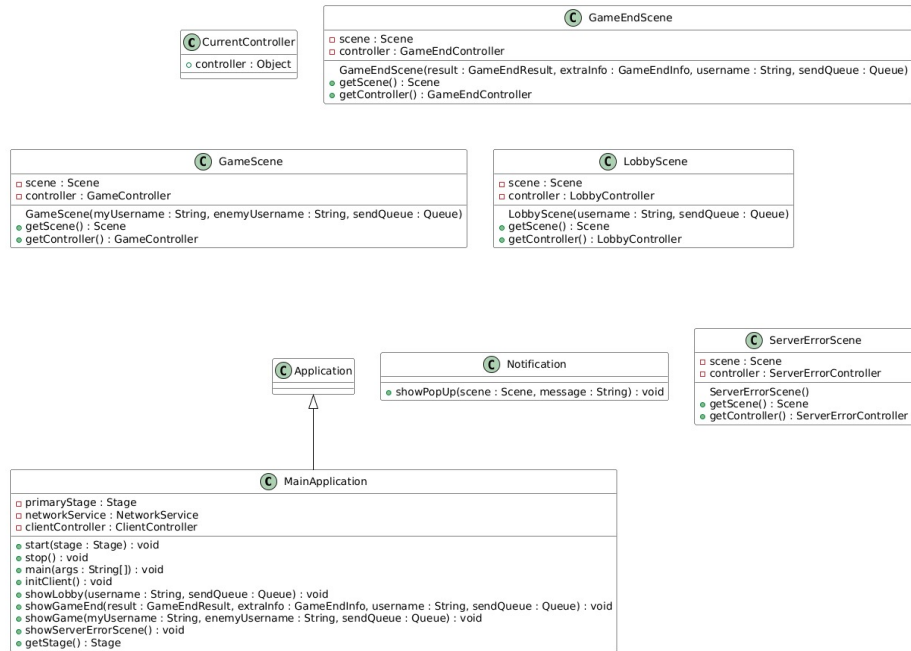
8.2.1 Package client



8.2.2 Package data



8.2.3 Package application



8.2.4 Package application.controllers

GameController
- grid : GridPane - opponentNameLabel : Label - turnStatusLabel : Label - sendQueue : Queue - lastMove : int - ROWS : int - COLS : int - cells : StackPane[[]] - gameFinished : boolean - myTurn : boolean - myColor : Color - otherColor : Color - myUsername : String - enemyUsername : String
- initialize() : void - updateGameStatus(isMyTurn : boolean) : void - createGrid() : void - highlightColumn(col : int, highlight : boolean) : void - setQueue(sendQueue : Queue) : void - newMove(column : int) : void - handleInvalidMove() : void - handleNotYourTurn() : void - isColumnFree(column : int) : boolean - placeYourMove() : void - placeEnemyMove(column : int) : void - dropDisc(col : int, color : Color) : void - setMyUsername(myUsername : String) : void - setEnemyUsername(enemyUsername : String) : void - getMyUsername() : String - setSendQueue(sendQueue : Queue) : void - setYourMove(move : Boolean) : void - showGameEnded() : void

GameEndController
- username : String - sendQueue : Queue
- setResults(result : GameEndResult, extraInfo : GameEndInfo) : void - handleBackToLobby() : void - setUsername(username : String) : void - setSendQueue(sendQueue : Queue) : void

LobbyController
- sendQueue : Queue - nameSended : boolean - gameAccepted : boolean - username : String - newUsername : String
- initialize() : void - setUsername(username : String) : void - setSendQueue(sendQueue : Queue) : void - addPlayerToUI(name : String, status : String) : void - handleRefreshPlayers() : void - addPlayers(players : ArrayList<Player>) : void - clearPlayersUI() : void - sendRequest(playerName : String) : void - handleClientRequestedNotFound(username : String) : void - removeAllSentRequest() : void - handleResultDeclined(opponentName : String) : void - addIncomingRequest(opponentName : String) : void - removeIncomingRequest(opponentName : String) : void - removeAllIncomingRequest() : void - handleDeclineRequest(opponentName : String) : void - handleAcceptRequest(opponentName : String) : void - handleGameJustAccepted() : void - handleUpdateName() : void - updateUsername() : void - handleUsernameTaken() : void - handleNameSended() : void - getUsername() : String

ServerErrorController
handleCloseApp() : void

9 Repository GitHub del progetto

<https://github.com/Donatipaolo/Forza-Quattro>

10 Conclusioni

Con questo progetto abbiamo acquisito una comprensione più approfondita della programmazione di sistemi client-server. In particolare, abbiamo analizzato il processo di definizione di un protocollo di comunicazione e compreso l'importanza di avere un protocollo proprietario per garantire uno scambio di messaggi chiaro tra client e server.

Abbiamo inoltre studiato il funzionamento delle architetture client e server, osservando come un server multithread sia in grado di gestire simultaneamente più connessioni, consentendo a diversi giocatori di interagire e disputare partite nello stesso momento. Infine, lo sviluppo del client ci ha permesso di comprendere l'importanza di una struttura software suddivisa in moduli: separare la gestione della rete, la logica del gioco e l'interfaccia grafica contribuisce a rendere il codice più leggibile e chiaro.